

บทที่ 3

การพัฒนา web ด้วย XML

XML ได้ถูกรองรับโดยองค์กร W3C ให้เป็น มาตรฐานสำหรับการแลกเปลี่ยนข้อมูล เมื่อวันที่ 10 กุมภาพันธ์ 1998 โดย XML ถูกออกแบบมาสำหรับการเก็บบันทึกข้อมูล แต่ไม่ได้ใช้แทนที่ HTML

ดังที่กล่าวมาแล้วในบทที่ 2 ว่า XML พัฒนาเพื่อปรับปรุงข้อด้อยของ HTML ที่ไม่ยืดหยุ่น และไม่สามารถตอบสนองการใช้งานข้ามแพลตฟอร์มได้อย่างแท้จริง แต่จุดประสงค์ของ XML และ HTML ก็ยังคงต่างกัน นั่นคือ XML ถูกออกแบบมาเพื่ออธิบายข้อมูลในเอกสาร ซึ่งเน้นไปในเรื่องของข้อมูลนั้นเกี่ยวกับอะไร ส่วน HTML ออกแบบมาเพื่อการแสดงผลข้อมูล และเน้นไปในเรื่องของผลลัพธ์ในการแสดงผลข้อมูลว่าจะมีหน้าตาอย่างไรมากกว่า นั่นคือ W3C คาดหวังให้การพัฒนา Web ในอนาคตนั้นหันมาใช้ XML แต่ใช้เพื่ออธิบายข้อมูล ในขณะที่ยังคงต้องอาศัย HTML ในการแสดงผลข้อมูลนั้น

แล้วทำไมจึงควรใช้ XML ในการพัฒนา web application ?

นอกจาก XML จะมีความยืดหยุ่นอย่างมากแล้ว XML ใช้หลักการแยกส่วนของข้อมูลและการแสดงผลออกจากกัน โดยเรียกวิธีการโปรแกรมโดยที่แยกข้อมูลออกจากส่วนของการแสดงผลนี้ว่า Content/Presentation paradigm ซึ่งวิธีนี้มีประโยชน์ในการที่เราสามารถเปลี่ยนแปลงการนำเสนอข้อมูลให้เหมาะสมกับสื่อหรือแพลตฟอร์มต่าง ๆ ได้ตามที่เราต้องการโดยไม่กระทบกับข้อมูลดั้งเดิมทำให้มีประโยชน์อย่างมากในการจัดการข้อมูล และช่วยให้เราสามารถส่งข้อมูลที่อยู่ในรูป XML ไปใช้บนแพลตฟอร์มใด ๆ ก็ได้

หากคุณต้องติดต่อกับบริษัทอื่น ๆ และจำเป็นต้องแชร์ใช้ข้อมูลร่วมกัน ก็สามารถเก็บข้อมูลในรูปแบบของ XML และแสดงผลออกมาบนแบบฟอร์มของบริษัทของตนเองบนแอปพลิเคชันของตนเอง โดยไม่ต้องเข้าไปแก้ไขข้อมูลหรือกรอกข้อมูลใหม่ให้อยู่ในรูปแบบฟอร์มที่ตรงกัน การใช้งานบน web ก็เช่นกัน หากเราจัดทำข้อมูลในรูปแบบ XML นอกจากง่ายต่อการจัดการเพิ่มเติม แก้ไข ข้อมูล แล้ว เราสามารถเปลี่ยนแปลงให้การแสดงผลให้อยู่ในรูปแบบใดก็ได้ โดยไม่ส่งผลกระทบต่องานข้อมูล นอกจากนี้เรายังสามารถนำข้อมูลเดียวกันนี้มาแสดงในรูปแบบ HTML บน web หรือส่งต่อเป็นเอกสาร .pdf หรือแสดงผลข้อมูลในรูปแบบใด ๆ ก็ได้ที่คุณต้องการ ซึ่งขึ้นอยู่กับงานเขียนส่วนการแสดงผลของเอกสาร โดยไม่จำเป็นต้องกลับไปแก้ไข หรือยุ่งเกี่ยวกับไฟล์ข้อมูลเลย ในทางกลับกัน เราสามารถแก้ไขข้อมูลในเอกสาร XML ซึ่งทำให้สามารถ update การแสดงผลบน web ได้โดยไม่ต้องคอยแก้ code HTML ก็สามารถทำได้

เมื่อเราทราบแล้วว่า XML ใช้งานโดยแยกส่วนของข้อมูลออกจากการแสดงผล และส่วนของข้อมูลก็คือเอกสาร .xml แล้วจะนำเอกสาร XML นี้มาแสดงผลได้อย่างไร

XML นั้นมีตัวช่วยเพื่อจัดการในส่วนของการแสดงผลหลายแบบ แต่สำหรับการแสดงผลเป็น web นั้น ใช้สไตลชีตเข้ามาช่วยจัดการในส่วนของการแสดงผล โดยสไตลชีตที่สามารถนำมาใช้แสดงผล XML หรือที่เกี่ยวข้องกับการนำ XML มาแสดงผลนั้นมีหลายชนิด เช่น CSS (Cascading Style Sheet), XSL (XML Style Sheets), XSLT (XSL Transformations) เป็นต้น แต่ CSS นั้นไม่ใช่ สไตลชีตที่คิดค้นมาสำหรับ XML โดยเฉพาะ เหมือน XSL และ XSLT ดังนั้นสำหรับโครงการนี้จึงสนใจการใช้งาน XML ร่วมกับ XSL เป็นหลัก

3.1 XSL Languages

XSL ย่อมาจาก EXtensible Stylesheet Language ซึ่งองค์กร W3C ได้พัฒนาขึ้นเพื่อให้เป็นภาษาที่มีลักษณะแบบ XML ที่ใช้ในการสร้างสไตลชีต

เนื่องจาก tag ใน XML เป็น tag ที่ผู้ใช้สามารถกำหนดได้เอง ไม่ใช่ tag ที่ได้กำหนดความหมายเอาไว้แล้วเหมือนอย่างใน HTML เช่น อิลิเมนต์ <table> อาจจะหมายถึงตารางใน HTML แต่สำหรับ XML แล้วอาจหมายถึงโต๊ะ หรืออะไรก็ได้ ดังนั้น browser จึงไม่รู้แสดงผลมันอย่างไร จึงจำเป็นต้องมี XSL เข้ามาอธิบายว่าจะให้ เอกสาร XML นี้แสดงผลเป็นอย่างไร

กลไกของ XSL ใช้สไตลชีตเพื่อแปลงเอกสาร XML ทั้งหมดไปเป็นเอกสารชนิดอื่น ๆ และเพื่อจัดรูปแบบ output ภายในสไตลชีตเหล่านี้ เราต้องกำหนดเลย์เอาต์ของเอกสารเอาไว้ รวมทั้งที่จะได้ข้อมูลมาใช้ในเอกสาร input ก็เหมือนกับการบอกว่า “ข้อมูลที่คุณต้องการ อยู่ ณ จุดนี้ในเอกสาร input และให้ทำให้มีลักษณะแบบนี้ในเอกสาร output นะ” นั่นเอง ภาษาเฉพาะของ XSL จะเรียกเอกสาร input ว่า ต้นไม้ต้นฉบับ (source tree) และเรียกเอกสาร output ว่า ต้นไม้ผลลัพธ์ (result tree)

XSL ประกอบด้วย 3 ส่วนสำคัญได้แก่

- XSLT เป็นภาษาสำหรับแปลงเอกสาร XML ให้เป็นเอกสารในฟอร์แมตอื่น ๆ ได้ เช่น ถ้าต้องการใช้แสดงผลบนอินเทอร์เน็ตก็ให้แปลงเอกสาร XML ให้เป็นเอกสาร XHTML เพื่อแสดงผลได้
- XPath ทำหน้าที่ชี้ตำแหน่ง (address) ของส่วนต่าง ๆ ในเอกสาร XML มีประโยชน์ในการค้นหาข้อมูลต่าง ๆ จากเอกสาร XML
- XSL-FO เป็นภาษาสำหรับกำหนดรูปแบบ (formatting) ให้กับเอกสาร XML เพื่อการแสดงผล ให้มีหน้าตาให้สวยงามอย่างที่เราต้องการ ส่วนใหญ่ใช้จัดหน้าเอกสาร PDF

เนื่องจาก XSLT ได้ถูกออกแบบมาให้ทำงานได้ด้วยตนเอง (self-standing) นั่นคือ นักพัฒนาสามารถเขียนกลไกของ XSLT ขึ้นมาได้โดยไม่ต้องมีฟังก์ชันของ XSL-FO อยู่ด้วย และเนื่องจากโครงงานนี้เป็นการพัฒนา web จึงเน้นการศึกษาและใช้งานในส่วนของ XSLT และ XPath มากกว่า

3.2 XSLT

XSLT คือ XSL Transformations ใช้สำหรับแปลงเอกสาร XML ให้เป็นเอกสาร XML ในรูปแบบอื่น ๆ (มีประโยชน์ในการแลกเปลี่ยนข้อมูลอย่างมาก เช่นแบบฟอร์มสั่งซื้อสินค้าที่เป็นเอกสาร XML จากต่าง ๆ บริษัทกัน ย่อมมีรูปแบบไม่เหมือนกัน ก็สามารถแปลงให้อยู่ในรูปแบบเดียวกันเพื่อให้บริษัทที่รับ order จากบริษัทต่าง ๆ นั้นเข้าใจได้ง่ายขึ้น) หรือแปลงให้เป็นเอกสารประเภทอื่นที่ browser เข้าใจ เช่น HTML หรือ XHTML ได้ ซึ่งโดยปกติแล้ว XSLT มักแปลงแต่ละอิลิเมนต์ของ XML ให้อยู่ในรูปอิลิเมนต์ของ XHTML

ด้วย XSLT เราสามารถเพิ่มหรือลบอิลิเมนต์และแอตทริบิวต์ ที่เกี่ยวข้องกับไฟล์ output เราสามารถจัดการเรียงลำดับหรือ sort อิลิเมนต์ได้ใหม่ เลือกให้แสดงหรือซ่อนบางอิลิเมนต์ได้ตามที่เราต้องการ โดย XSLT จะใช้ XPath ในการค้นหาข้อมูลที่ต้องการจากเอกสาร XML เมื่อ XPath พบข้อมูลที่ต้องการ XSLT ก็จะทำการแปลงส่วนที่ต้องการนั้นจากเอกสารต้นฉบับให้เป็นเอกสารผลลัพธ์ที่ต้องการ

XSLT ได้กลายมาเป็น W3C Recommendation ตั้งแต่วันที่ 16 พฤศจิกายน 2541 แต่ก็ไม่ใช่ว่า Internet browser ทุกตัวที่รองรับ XML และ XSLT อย่างเต็มตัว โดยความสามารถในการรองรับ XML และ XSLT ของ Internet browser ในปัจจุบัน ดังแสดงในตารางที่ 3.1 (ที่มา: <http://www.w3schools.com/xsl>)

ตารางที่ 3.1 ความสามารถในการรองรับ XML และ XSLT ของ Internet browser ในปัจจุบัน

Internet browser	XML และ XSLT support
Firefox 1.0.2	สนับสนุน XML, XSLT และ CSS
Mozilla 1.7.8	include Expat สำหรับ XML parsing และสนับสนุนการแสดงผลด้วย XML + CSS และสนับสนุน Namespaces อยู่บ้าง
Netscape 8	มีพื้นฐานมาจาก Mozilla จึงสนับสนุน XML เหมือน Mozilla
Opera 8	สนับสนุน XML + CSS
Internet Explorer 5	การสนับสนุน W3C XSL Recommendation ยังไม่สมบูรณ์

ตารางที่ 3.1 (ต่อ) ความสามารถในการรองรับ XML และ XSLT ของ Internet browser ในปัจจุบัน

Internet browser	XML และ XSLT support
Internet Explorer 6	สนับสนุน XML อย่างเต็มที่ (Full XML support) รวมไปถึงสนับสนุน Namespaces, CSS, XSLT 1.0 MSXML Parser 3.0 ใน Internet Explorer 6 และ Windows XP นั้นมีพื้นฐานมาจากทั้ง W3C XSLT Recommendation 1.0 และ W3C XPath 1.0 Recommendation

การใช้งาน XSL หรือ XSLT นั้น ประกอบด้วย 2 ส่วน คือ ส่วนของเอกสาร XML และเอกสาร XSLT สำหรับแปลงเอกสาร XML ดังกล่าว

ในการเริ่มสร้างเอกสาร XSLT นั้นเราต้องทำการประกาศ root element สำหรับเอกสาร XSL style sheet โดยสามารถใช้ <xsl:stylesheet> หรือ <xsl:transform> ก็ได้ ในตัวอย่างที่ 3.1 แสดงวิธีการประกาศ XSL style sheet ที่ถูกต้องตาม W3C XSLT Recommendation โดยใช้อิเลิเมนต์รากทั้งสองแบบ

ตัวอย่างที่ 3.1 การประกาศ XSL style sheet ที่ถูกต้อง

<pre><xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"> หรือ <xsl:transform version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"></pre>
--

ในการใช้งาน XSLT element, attributes และคุณสมบัติต่าง ๆ ของมัน เราต้องประกาศ XSLT namespace ไว้บนสุดของเอกสารด้วย โดย xmlns:xsl="http://www.w3.org/1999/XSL/Transform" ซึ่งไปยัง official W3C XSLT namespace ซึ่งในการใช้ namespace นี้ คุณต้อง include แอตทริบิวต์ version="1.0" ด้วยเสมอ

สำหรับการ link เอกสาร XSL ที่ต้องการใช้ในการแปลงเอกสาร XML นั้น เราต้องเพิ่มอิเลิเมนต์ `<?xml-stylesheet type="text/xsl" href="***.xsl"?>` ในเอกสาร XML นั้น โดยระบุชื่อไฟล์ XSL ที่ต้องการใช้แปลง พิจารณาตัวอย่างที่ 3.2 แสดงเอกสาร service_xxx.xml ซึ่งอ้างอิงถึง libraryinfo.xsl สำหรับแปลงเอกสาร

ตัวอย่างที่ 3.2 การเพิ่ม XSL style sheet reference ลงในเอกสาร "sevice_xxx.xml"

```
<?xml version="1.0" encoding="UTF-8"?>
<?DOCTYPE service SYSTEM "libraryinfo.dtd" ?>

<?xmlstylesheet type="text/xsl" href="libraryinfo.xsl" ?>

<libraryinfo>
    .....
</libraryinfo>
```

เมื่อเราทราบวิธีการเชื่อมโยงเอกสาร XSL เข้ากับเอกสาร XML แล้ว หัวข้อต่อไปนี้จะกล่าวถึง อิลิเมนต์ต่าง ๆ ที่ใช้สร้างเอกสาร XSLT โดยตารางที่ 3.2 แสดง XSLT element รวมทั้งความหมาย ของการใช้งานอย่างคร่าว ๆ และเวอร์ชันที่รองรับการใช้งานใน Internet Explorer (IE) และ Netscape (NN)

ตารางที่ 3.2 ความหมายของ XSLT อิลิเมนต์ที่ใช้ในเอกสาร และความสามารถรองรับใน IE และ NN

Element	อธิบาย	IE	NN
apply imports	Applies template rule จากสไตร์ชีทที่ import มา	6.0	
apply templates	Applies template rule ไปยังอิลิเมนต์ปัจจุบันหรือไปยังลูกหลาน ของอิลิเมนต์ปัจจุบัน	5.0	6.0
attribute	เพิ่ม attribute	5.0	6.0
attribute-set	กำหนดกลุ่มของชื่อให้กับ attributes	6.0	6.0
call-template	Calls a named template	6.0	6.0
choose	ใช้ในการเชื่อมระหว่าง <when> และ <otherwise> เพื่อ express multiple conditional tests	5.0	6.0
comment	สร้าง comment node ใน result tree	5.0	6.0
copy	สร้าง copy ของ current node (ไม่รวมถึงโหนดลูกและ attributes)	5.0	6.0
copy-of	สร้าง copy ของ current node (โดยรวมถึงโหนดลูกและ attributes)	6.0	6.0

ตารางที่ 3.2 (ต่อ) ความหมายของ XSLT อิลิเมนต์ที่ใช้ในเอกสาร และความสามารถรองรับใน IE และ NN

Element	อธิบาย	IE	NN
decimal-format	Defines ตัวอักษรและสัญลักษณ์ที่จะใช้สำหรับการ convert ตัวเลขไปเป็น strings โดยใช้ format-number() function	6.0	
element	สร้างอิลิเมนต์ไหนในเอกสาร output	5.0	6.0
fallback	ระบุ code ที่จะ run แทน ถ้า processor ไม่ support XSLT element	6.0	
for-each	Loops เข้าไปในแต่ละไหนภายใต้ไหนที่กำหนด	5.0	6.0
if	บรรจุ template ที่จะให้ใช้ถ้าเงื่อนไขที่กำหนดไว้เป็นจริง	5.0	6.0
import	Imports contents ของสไตลชีทหนึ่งไปยังสไตลชีทอื่น ๆ Note: imported style sheet ถือว่ามี precedence (ความสำคัญ) ที่ต่ำกว่า importing style sheet	6.0	6.0
include	Includes contents ของสไตลชีทหนึ่งไปยังสไตลชีทอื่น ๆ Note: included style sheet มี precedence เท่ากับ including style sheet	6.0	6.0
key	ประกาศชื่อ key ที่สามารถใช้ในสไตลชีทด้วย key() function	6.0	6.0
message	เขียน message ลงใน output (มักใช้ในการรายงาน error)	6.0	6.0
namespace-alias	แทนที่ namespace ในสไตลชีทไปเป็นชื่อ namespace อื่นในเอกสาร output	6.0	
number	ตรวจสอบตำแหน่ง integer ของ current node และ formats ของเลขนั้น	6.0	6.0
otherwise	กำหนด default action สำหรับ <choose> element	5.0	6.0
output	กำหนด format เอกสาร output	6.0	6.0
param	ประกาศ local หรือ global parameter	6.0	6.0
preserve-space	กำหนด elements ที่ให้คง white space เอาไว้ตามต้นฉบับ	6.0	6.0
processing-instruction	เขียน processing instruction ลงใน output	5.0	6.0
sort	เรียงลำดับให้ผลลัพธ์	6.0	6.0
strip-space	กำหนด elements ที่ให้ตัด white space ทิ้งไป	6.0	6.0
stylesheet	กำหนด root element ของ style sheet	5.0	6.0

ตารางที่ 3.2 (ต่อ) ความหมายของ XSLT อิลิเมนต์ที่ใช้ในเอกสาร และความสามารถรองรับใน IE และ NN

Element	อธิบาย	IE	NN
template	บรรจุกฎที่จะใช้เมื่อพบโหนดที่ตรงกับโหนดที่ระบุไว้	5.0	6.0
text	เขียนตามตัวอักษรอย่างแท้จริงออกไปยัง output	5.0	6.0
transform	กำหนด root element ของ style sheet	6.0	6.0
value-of	ดึงเอาค่า (value) ของโหนดที่เลือกออกมาแสดงผล	5.0	6.0
variable	ประกาศ local หรือ global variable	6.0	6.0
when	ระบุ action สำหรับ <choose> element	5.0	6.0
with-param	กำหนดค่าของ parameter ที่ส่งต่อไปยัง template	6.0	6.0

เนื่องจากอิลิเมนต์ภายใต้ข้อกำหนดของ XSLT 1.0 มีจำนวนมาก จึงจำเป็นต้องเลือกเพียงบางอิลิเมนต์ที่สำคัญและใช้งานบ่อย มาอธิบายรายละเอียด วิธีใช้งาน และแอตตริบิวต์ภายใต้อิลิเมนต์นั้น ในหัวข้อย่อยต่อไปนี้จะสำหรับรายละเอียดของอิลิเมนต์อื่น ๆ สามารถศึกษาเพื่อเพิ่มเติมได้จาก <http://w3schools.com/xsl>, <http://www.w3.org/TR/xslt> หรือ <http://www.xslt.com>

3.2.1 <xsl:template> element

XSLT ถูกสร้างขึ้นมาจากโครงสร้างที่เรียกว่า เทมเพลต (Template) เทมเพลตจะเป็นตัวกำหนดสิ่งที่ต้องค้นหาจากต้นไม้มันฉบับ และสิ่งที่ต้องใส่เข้าไปในต้นไม้ผลลัพธ์

ใน XSLT เทมเพลต ถูกกำหนดขึ้นโดยอิลิเมนต์ <xsl:template> ที่มี 2 ส่วนสำคัญคือ แอตตริบิวต์ match และเนื้อหาของเทมเพลต โดยแอตตริบิวต์ match ใช้เชื่อมโยงเทมเพลต กับเอกสาร XML โดยแอตตริบิวต์ match จะเป็นสิ่งที่บอกว่าเทมเพลตภายใต้ <xsl:template> ที่กำหนด จะใช้กับเอกสาร XML ในส่วนไหน โดยค่าของแอตตริบิวต์ match คือ XPath Expression เช่น match="/" อ้างถึงเอกสารทั้งหมด

Syntax

```
<xsl:template
  name="name"
  match="pattern"
  mode="mode"
  priority="number">
  <!-- Content:(<xsl:param>*,template) -->
</xsl:template>
```

Attributes

Attribute	Value	ความหมาย
name	name	ใช้กำหนดชื่อให้กับ template (เลือกมีหรือไม่มีก็ได้) Note: ถ้าไม่มี attribute นี้ ต้องมี match attribute
match	pattern	กำหนด match pattern สำหรับ template (เลือกมีหรือไม่มีก็ได้) Note: ถ้าไม่มี attribute นี้ ต้องมี name attribute
mode	mode	ระบุ mode สำหรับ template นี้ (มีหรือไม่มีก็ได้)
priority	number	เลข number แสดงถึงค่าความสำคัญแบบ numeric สำหรับ template (มีหรือไม่มีก็ได้)

3.2.2 <xsl:value-of > element

<xsl:value-of > สามารถนำเอาข้อมูล PCDATA จากอิลิเมนต์ที่เลือก (โดยแอตตริบิวต์ select) มาแสดงผลบนต้นไม้ผลลัพธ์ได้

Syntax

```
<xsl:value-of select="expression" disable-output-escaping="yes|no" />
```

Attributes

Attribute	Value	ความหมาย
select	expression	ใส่ XPath expression เพื่อระบุว่า node/attribute ไหนที่ต้องการให้ดึง value ออกมา
disable-output-escaping	yes no	"yes" หมายถึงการอนุญาตให้ special characters (เช่น "<") แสดงผลได้ตามนั้น "no" เป็นการกำหนดให้ special characters (เช่น "<") กลายเป็น "<" เมื่อออกเป็น output จะกำหนดค่านี้หรือไม่มีก็ได้ ถ้าไม่ระบุถือว่ามีค่า "no" เสมอ

3.2.3 <xsl:apply-templates> Element

<xsl:apply-templates> element ถูกนำมาใช้ภายในเท็มเพลตใดเท็มเพลตหนึ่งเพื่อเรียกอีกเท็มเพลตหนึ่ง เมื่อเราเพิ่ม select attribute ลงใน <xsl:apply-templates> element มันจะทำงานเฉพาะไหนดลูกที่ตรงกับค่าของแอตตริบิวต์ เราสามารถใช้ select attribute เพื่อระบุว่าจะให้ไหนดลูกแสดงผลเป็นอย่างไร

Syntax

```
<xsl:apply-templates
  select="expression"
  mode="name">
  <!-- เพิ่มเพลตที่ต้องการ -->
</xsl:apply-templates>
```

Attributes

Attribute	Value	ความหมาย
select	expression	(เลือกได้) ระบุ nodes ที่ต้องการประมวลผล ถ้าแทนด้วยเครื่องหมายดอกจัน (*) จะเลือก node-set ทั้งหมด ถ้าไม่ได้ attribute จะเลือกโหนดลูกของ current node ทุกโหนด
mode	name	(เลือกได้) ถ้าเรามีหลายวิธีในการจัดการกับอิลิเมนต์เดียวกัน เราควรกำหนดชื่อให้กับอิลิเมนต์ <xsl:apply-templates> เพื่อสะดวกต่อการเรียกใช้งาน

3.2.4 <xsl:for-each> Element

<xsl:for-each> element จะ loops เข้าไปในแต่ละโหนดของ node set ที่กำหนด

Syntax

```
<xsl:for-each select="expression">
  <!-- คำสั่งอื่นๆ -->
</xsl:for-each>
```

Attributes

Attribute	Value	ความหมาย
select	expression	ต้องมีเสมอ โดยเป็น XPath ระบุ node set ที่ทำการ loop เข้าไป สำหรับ filter operation ที่ใช้ได้แก่ = เท่ากับ != ไม่เท่ากับ < น้อยกว่า > มากกว่า

3.2.5 <xsl:sort> Element

<xsl:sort> element ใช้ sort ผลลัพธ์ตามที่ แอตทริบิวต์ select โดย <xsl:sort> ต้องใช้ คู่กับ <xsl:for-each> หรือ <xsl:apply-templates> เสมอ

Syntax

```
<xsl:sort
  select="expression"
  lang="language-code"
  data-type="text|number|qname"
  order="ascending|descending"
  case-order="upper-first|lower-first"/>
```

Attributes

Attribute	Value	ความหมาย
select	XPath-expression	(เลือกได้) ใช้ระบุว่าจะให้ sort ตาม node/node-set ไหน
lang	language-code	(เลือกได้) ระบุว่าจะให้เรียงตามแบบของภาษาอะไร
data-type	text number qname	(เลือกได้) ระบุ data-type ของข้อมูลที่จะถูกเรียง โดย Default คือ "text"
order	ascending descending	(เลือกได้) ระบุรูปแบบการเรียง (sort order) Default คือ "ascending" (ค่าน้อยไปมาก)
case-order	upper-first lower-first	(เลือกได้) ระบุว่าจะระหว่างตัวพิมพ์เล็กกับตัวพิมพ์ใหญ่จะให้เรียงลำดับโดยให้อะไรมาก่อน

3.2.6 <xsl:if> Element

<xsl:if> element บรรจ template ที่จะนำมาใช้เมื่อ condition เป็นจริง

ข้อแนะนำ : สามารถใช้ <xsl:choose> ในสัณฐานร่วมกับ <xsl:when> และ <xsl:otherwise> ในการแสดงผลโดยใช้หลาย ๆ เงื่อนไข

Syntax

```
<xsl:if test="expression">
    <!-- Content: template -->
</xsl:if>
```

Attributes

Attribute	Value	ความหมาย
test	expression	ต้องมีเสมอ ใช้ระบุเงื่อนไขที่ต้องการทดสอบ

นอกจากอิลิเมนต์ดังที่กล่าวมาแล้ว XSLT 2.0, XPath 2.0 และ XQuery 1.0 ได้มีการใช้ฟังก์ชัน library ร่วมกัน โดย XSLT นั้นได้ include มากกว่า 100 built-in functions โดยเป็นฟังก์ชันสำหรับ values, numeric values, วันและเวลา, node และการจัดการ QName, sequence manipulation, Boolean values และอื่น ๆ โดยตาราง 3.3 ได้แสดงฟังก์ชันที่ควรทราบ และสามารถทำความรู้จักกับฟังก์ชันอื่น ๆ เพิ่มเติมได้จาก <http://www.w3.org/2005/02/xpath-functions>

สำหรับ default prefix สำหรับฟังก์ชัน namespace คือ "fn:"

ข้อเสนอแนะ : ฟังก์ชันมักถูก call ด้วย fn: prefix เช่น fn:string() อย่างไรก็ตาม เมื่อ fn: กลายเป็น default prefix ของ namespace แล้วชื่อของฟังก์ชันก็ไม่จำเป็นต้องเติม prefix อีกเมื่อถูก call

ตารางที่ 3.3 ตัวอย่าง built-in function ของ XSLT

ชื่อฟังก์ชัน	ความหมาย
current()	Returns ค่าของ current node
document()	ใช้เข้าถึงโหนดในเอกสาร XML ภายนอก
element-available()	ทดสอบว่าอิลิเมนต์ที่ระบุนั้น สามารถรองรับโดย XSLT processor หรือไม่
format-number()	Converts ตัวเลขไปเป็น string
function-available()	ทดสอบฟังก์ชันที่ระบุว่าจะสามารถรองรับโดย XSLT processor ได้หรือไม่
generate-id()	Returns ค่า string value ที่เป็น uniquely identifies สำหรับโหนดที่ระบุ
key()	Returns ค่า node-set ใช้ระบุ index โดย <xsl:key> element
system-property()	Returns ค่าของ system properties
unparsed-entity-uri()	Returns ค่า URI ของ unparsed entity

สำหรับการเรียกใช้ XSLT ใช้แปลงเอกสาร XML สามารถกระทำบนฝั่ง Client หรือบนฝั่ง Server ก็ได้ โดยเราสามารถใช้อำนาจโปรแกรมมิ่งอื่น ๆ เข้าช่วยในการบอกให้แปลงเอกสารบนฝั่งไหน โดยแต่ละวิธีก็มีข้อดีและข้อเสียที่แตกต่างกัน

โดยถ้าทำการแปลงเอกสาร XML ด้วย XSLT บนฝั่ง server มักใช้ภาษา ASP, PHP, JSP, Python ฯลฯ มาช่วยในการแปลงเอกสาร ซึ่งทำให้ webpage ที่ได้ มีความ dynamic ขึ้นจากภาษาดังกล่าว และช่วยให้เราแน่ใจได้ว่า user จะสามารถเปิดเอกสารดูได้จาก browser ใด ๆ ก็ได้ แม้ว่า browser นั้นจะไม่รองรับการทำงานของ XSLT ก็ตาม เนื่องจาก เราได้ให้ภาระในการแปลงเอกสารอยู่ที่ server แล้วจึงค่อยส่งไฟล์ที่แปลงแล้ว (เช่น .html, .php) ส่งไปให้ user

อีกวิธีคือทำการแปลงเอกสารบนฝั่ง Client โดยการให้ user download เอกสาร XML และ XSLT และให้ browser ที่ฝั่ง user เป็นผู้แปลงโดยตรง หรือจะแปลงผ่าน JavaScript ก็สามารทำได้ ซึ่งข้อเสียของวิธีนี้คือ ถ้า browser ของ user ไม่รองรับการทำงานของ XSLT จะทำให้ไม่สามารถแปลงเอกสารดังกล่าวได้ แต่ในปัจจุบัน XSLT ได้กลายมาเป็นมาตรฐานสำหรับ browser ต่าง ๆ โดยเฉพาะ IE 6 และ FireFox ที่เป็นที่ยอมรับ ล้วนรองรับการทำงานของ XML อย่างเต็มที่ ทำให้ปัญหาของการที่ browser ไม่รองรับ นับว่ามีผลน้อยมาก (เป็นส่วนน้อยจริง ๆ ที่ user ไม่ได้ upgrade browser ให้ทันสมัย) เมื่อเปรียบเทียบกับข้อเสียของการแปลงเอกสารบนฝั่ง server เนื่องจาก การแปลงเอกสารบนฝั่ง server นั้น ทำให้สิ้นเปลืองทรัพยากรและ server ทำงานหนักมากเกินไป เมื่อคำนึงถึงกรณีที่ว่าหากมี user จำนวนมากต้องการเข้าถึง web ที่อยู่บน server นั้นในช่วงเวลาหนึ่งพร้อม ๆ กัน แทนที่ server จะใช้ทรัพยากรไปในการตอบสนองการร้องขอจาก user กลับต้องมาใช้โปรเซสเซอร์ในการแปลงเอกสารอีก ซึ่งอาจส่งผลให้มีการตอบสนองช้าลงมาก ซึ่งส่งผลต่อปริมาณ traffic ในเครือข่าย และส่งผลกระทบต่ออื่น ๆ เป็นวงกว้างได้ โดยเฉพาะเมื่อโครงการนี้จะนำไปใช้บน server ของสถาบัน จึงควรคำนึงถึงประเด็นนี้มากกว่า ดังนั้น สำหรับโครงการนี้ จึงได้เลือกใช้การแปลงเอกสารบนฝั่ง Client โดยใช้การแปลงเอกสาร XML ด้วย XSLT โดยตรงเป็นหลัก ซึ่งจะอธิบายแนวคิดและรูปแบบการทำงานของเอกสาร XML ร่วมกับ XSL สำหรับระบบแนะนำหอสมุดกลางบนเว็บ ในบทที่ 5

จากหัวข้อ XSLT โปรดสังเกตว่า คำสำหรับแอดดรีวิวิต์ ในหลาย ๆ อิลิเมนต์ของ XSLT นั้นใช้ XPath Expression ในการระบุโหนดที่ต้องการ ในหัวข้อต่อไปจะเป็นการแนะนำหลักการของ XPath ที่ถูกต้อง ซึ่งเป็นอีกส่วนสำคัญของ XSL

3.3 XPath

XPath เป็นภาษาสำหรับใช้ค้นหาข้อมูลในเอกสาร XML โดยใช้การท่องไปตามอิลิเมนต์และแอตทริบิวต์ในเอกสาร XML

โดย XPath เป็นส่วนที่คอยบอกตำแหน่งว่าตอนนี้เราอยู่ที่ไหน และจะไปที่ไหนในเอกสารที่ละชั้น แต่การที่เราจะรู้ว่าเราอยู่ ณ ที่ใดของเอกสาร XPath จำเป็นต้องรู้จัก context node ซึ่งคือส่วนของเอกสาร XML ที่เรากำลังตั้งต้นอยู่เสียก่อน

โหนด (node) ใน XPath คือ ส่วนใด ๆ ก็ตาม ที่อยู่ในเอกสาร ซึ่งแบ่งออกเป็น 7 แบบ ได้แก่ element, attribute, text, namespace, processing- instruction, comment และ document (root) node โดยโหนดเหล่านี้ที่อยู่ใน XML จะมีลักษณะแบบโครงสร้างต้นไม้ ซึ่งถ้าเรามีเอกสาร XML ดังนี้

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<bookstore>
<book>
    <title lang="en">Harry Potter</title>
    <author>J K. Rowling</author>
    <price>29.99</price>
</book>
</bookstore>
```

ตัวอย่างของโหนดในเอกสารดังกล่าวสามารถแสดงได้ดังนี้

```
<bookstore> (คือ document node)
<author>J K. Rowling</author> (คือ element node)
lang="en" (คือ attribute node)
```

โดยคำศัพท์อื่นที่ควรทราบคือ Atomic value คือโหนดที่ไม่มี โหนดลูก หรือ พ่อ เช่น J K. Rowling หรือ “en”

ในการทำงานของ XPath จะใช้โหนดเงื่อนไข (term node) เพื่ออ้างถึงส่วนต่าง ๆ ของอิลิเมนต์ (รวมถึง แอตทริบิวต์ หรืออะไรก็ตาม) ซึ่งเมื่อพบโหนดที่ตรงตามเงื่อนไขแล้ว XPath ก็จะเป็นค่าสิ่งที่เราต้องการออกมา ถ้าหากบางครั้งเราใช้เงื่อนไขเป็นแอตทริบิวต์ที่มีปรากฏอยู่ในหลาย ๆ อิลิเมนต์ เช่น ให้นำอิลิเมนต์ใด ๆ ที่มีแอตทริบิวต์ lang แล้วได้ผลลัพธ์หลายอิลิเมนต์ XPath ก็จะเป็นค่ากลับมาเป็น โหนดเซต (node-set) ซึ่งเป็นคอลเลกชันของอิลิเมนต์ทั้งหมดในต้นไม้ของเอกสารที่มีแอตทริบิวต์ lang อยู่

สำหรับคอนเซ็ปต์ของรากของเอกสาร (document root node) ของ XPath นั้น ไม่จำเป็นต้องเป็นอันเดียวกับอิลิเมนต์รากในเอกสาร XML เสมอไป เนื่องจากมันสามารถเป็นสิ่งอื่นใดที่อยู่ตอนต้น หรือตอนท้ายของเอกสาร หรือจะอยู่ก่อนหรือหลังอิลิเมนต์รากของเอกสารก็ได้ ตามแต่เราจะกำหนด เพราะ XPath เพียงแค่ต้องการรากของเอกสารเสมือน (virtual document root) ที่ทำหน้าที่เป็นจุดเริ่มต้นสำหรับการค้นหา ข้อมูลจาก node ลูก ภายใต้อัน root ที่เรากำหนดเท่านั้น โดยรากใน XPath จะถูกระบุแทนด้วย “/” ซึ่งทำให้ เพิ่มเพลตในเอกสาร XSLT จึงได้ถูกสั่งให้จับคู่กับ “/” เช่น `<xsl:template match="/">`

สำหรับ Syntax ของ XPath เราจะใช้ตัวอย่าง 3.3 เพื่ออธิบาย Syntax ของ XPath

ตัวอย่างที่ 3.3 ตัวอย่างเอกสาร XML สำหรับอธิบาย Syntax ของ XPath

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<bookstore>
  <book>
    <title lang="eng">Harry Potter</title>
    <price>29.99</price>
  </book>
  <book>
    <title lang="eng">Learning XML</title>
    <price>39.95</price>
  </book>
</bookstore>
```

3.3.1 การเลือก node (Select Nodes)

XPath ใช้ path expressions ในการเลือกโหนดที่ต้องการจากเอกสาร XML โดยโหนดจะถูกเลือกโดย path หรือขั้นตอน ดังตารางที่ 3.4 ซึ่งแสดงถึง path expressions ใช้อย่างน้อย

ตารางที่ 3.4 path expressions ที่ใช้บ่อยและ ความหมาย

Expression	ความหมาย
<i>nodename</i>	เลือก child nodes ทั้งหมดของ node ชื่อนั้น ๆ
/	เลือกจาก root node

ตารางที่ 3.4 (ต่อ) path expressions ที่ใช้บ่อยและ ความหมาย

Expression	ความหมาย
//	เลือก nodes ในเอกสารจาก current node ที่ match กับที่ระบุ ไม่ว่ามันจะอยู่ในส่วนไหน
.	เลือก current node
..	เลือก parent ของ current node
@	เลือก attributes

ตัวอย่างเช่น

bookstore = เลือกโหนดลูกทั้งหมดที่อยู่ภายใต้อีลิเมนต์ bookstore
 /bookstore = เลือก root element ชื่อ bookstore
 bookstore/book = เลือกทุก book element ที่เป็นโหนดลูกของ bookstore
 //@lang = เลือกทุกแอตทริบิวต์ที่ชื่อ lang

3.3.2 Predicates

ใช้สำหรับหาโหนดที่เฉพาะเจาะจงหรือโหนดที่บรรจุค่าที่เราเจาะจง โดย Predicates จะต้องอยู่ใน [] เสมอ สำหรับตัวอย่างในการใช้ ดังแสดงในตารางที่ 3.5

ตารางที่ 3.5 ตัวอย่างการใช้ Predicates และผลลัพธ์

Path Expression	ผลจากการเรียกใช้
/bookstore/book[1]	เลือก book element แรก ที่เป็นโหนดลูกของ bookstore element
/bookstore/book[last()]	เลือก book element สุดท้ายที่เป็นโหนดลูกของ bookstore element
/bookstore/book[last()-1]	เลือกโหนดสุดท้ายเพียง 1 โหนดของ book element ที่เป็นลูกของ bookstore element
/bookstore/book[position()<3]	เลือก book elements 2 โหนดแรก ที่เป็นลูกของ bookstore element
//title[@lang]	เลือกทุก title elements ที่มี attribute ชื่อ lang

ตารางที่ 3.5 (ต่อ) ตัวอย่างการใช้ Predicates และผลลัพธ์

Path Expression	ผลจากการเรียกใช้
//title[@lang='eng']	เลือกทุก title elements ที่มี attribute ชื่อ lang ที่มีค่าเท่ากับ 'eng'
/bookstore/book[price>35.00]	เลือกทุก book elements ของ bookstore element ที่มี price element ที่ค่ามากกว่า 35.00
/bookstore/book[price>35.00]/title	เลือกทุก title elements ของ book elements ของ bookstore element ที่มี price element ที่ค่ามากกว่า 35.00

3.3.3 Selecting Unknown Nodes

XPath สามารถเลือกโหนดที่ไม่เจาะจงได้ ดังตารางที่ 3.6 และ 3.7 แสดงความหมายของสัญลักษณ์และตัวอย่างการใช้

ตารางที่ 3.6 ความหมายของสัญลักษณ์ใน XPath

Wildcard	คำอธิบาย
*	จับคู่กับ element node ใด ๆ
@*	จับคู่กับ attribute node ใด ๆ
node()	จับคู่กับโหนดใด ๆ ที่มีชนิดใด ๆ ก็ได้

ตารางที่ 3.7 ตัวอย่างการใช้งาน Selecting Unknown Nodes

Path Expression	ผลลัพธ์
/bookstore/*	เลือกทุก child nodes ของ bookstore element
//*	เลือกทุก elements ในเอกสาร
//title[@*]	เลือกทุก title elements ที่มีแอตทริบิวต์ใด ๆ

3.3.4 เลือกทีละหลาย ๆ path

ใช้สัญลักษณ์ | ในการเลือก หลาย path ได้ ตัวอย่างเช่น //book/title | //book/price หมายถึง เลือกทุก title และ price element จากทุก ๆ book element

3.3.5 XPath Axis

axis จะอ้างถึง node-set ที่เกี่ยวข้องสัมพันธ์กับโหนดปัจจุบัน ดังแสดงในตารางที่ 3.8

ตารางที่ 3.8 ชื่อ Axis และผลจากการใช้

AxisName	ผลลัพธ์
ancestor	เลือกบรรพบุรุษทั้งหมด (parent, grandparent, etc.) ของโหนดปัจจุบัน
ancestor-or-self	เลือกบรรพบุรุษทั้งหมด ของโหนดปัจจุบัน โดยเอาโหนดปัจจุบันด้วย
attribute	เลือกทุกแอตทริบิวต์ ของโหนดปัจจุบัน
child	เลือกโหนดลูกทั้งหมดของโหนดปัจจุบัน
descendant	เลือกโหนดลูกหลานทั้งหมด (children, grandchildren, etc.) ของโหนดปัจจุบัน
descendant-or-self	เลือกโหนดลูกหลานทั้งหมดของโหนดปัจจุบัน โดยเอาโหนดปัจจุบันด้วย
following	เลือกทุก ๆ อย่างในเอกสารที่อยู่หลังจากแท็กปิดของโหนดปัจจุบัน
following-sibling	เลือกทุก siblings หลังจาก current node
namespace	เลือกทุก namespace nodes ของ current node
parent	เลือกทุกโหนดพ่อของโหนดปัจจุบัน
preceding	เลือกทุก ๆ อย่างในเอกสารที่อยู่ก่อนจากแท็กเปิดของโหนดปัจจุบัน
preceding-sibling	เลือกทุก siblings ก่อน current node
self	เลือกโหนดปัจจุบัน

3.3.6 XPath Operators

ตารางที่ 3.9 แสดง XPath Operators ที่ใช้ได้ใน XPath และความหมาย

ตารางที่ 3.9 XPath Operators และความหมาย

Operator	ความหมาย	ตัวอย่าง	Return value
	Computes two node-sets	//book //cd	คืนค่า node-set ด้วยทั้ง book และ cd elements
+	บวก	6 + 4	10

ตารางที่ 3.9 (ต่อ) XPath Operators และความหมาย

Operator	ความหมาย	ตัวอย่าง	Return value
*	คูณ	6 * 4	24
div	หาร	8 div 4	2
=	เท่ากับ	price=9.80	true ถ้า price เป็น 9.80 false ถ้า price เป็น 9.90
!=	ไม่เท่ากับ	price!=9.80	true ถ้า price เป็น 9.90 false ถ้า price เป็น 9.80
<	น้อยกว่า	price<9.80	true ถ้า price เป็น 9.00 false ถ้า price เป็น 9.80
<=	น้อยกว่า หรือเท่ากับ	price<=9.80	true ถ้า price เป็น 9.00 false ถ้า price เป็น 9.90
>	มากกว่า	price>9.80	true ถ้า price เป็น 9.90 false ถ้า price เป็น 9.80
>=	มากกว่า หรือเท่ากับ	price>=9.80	true ถ้า price เป็น 9.90 false ถ้า price เป็น 9.70
or	หรือ	price=9.80 or price=9.70	true ถ้า price เป็น 9.80 false ถ้า price เป็น 9.50
and	และ	price>9.00 and price<9.90	true ถ้า price เป็น 9.80 false ถ้า price เป็น 8.50
mod	Modulus (หารเอาเศษ)	5 mod 2	1